## Maven ⭐ ⭐ ⭐ — 3–4 YOE Scenario-Based Q&A

**1. Maven build suddenly started failing, but it was working yesterday. What will you check?**
First, I check the failed Maven phase and console logs to understand the exact error. Then I compare the current build with the last successful build and check recent changes in code, pom.xml, dependencies, plugins, Java/Maven versions, repositories, or the Jenkins agent. Based on the error, I identify the root cause, fix it, and rerun the build.
**Simple ga gurthupettuko:**
Ninna work ayyi ivala fail ayithe → error ekkada vachindo chudu → last successful build tho compare cheyyi → recent change find chesi fix cheyyi.
**Eng:** Yesterday it worked and today it failed → check where the error occurred → compare with the last successful build → find the recent change and fix it.

---

**2. Maven dependency is not downloading. What will you check?**
First, I check the dependency name and version mentioned in pom.xml. Then I verify whether the dependency exists in the configured repository and check network connectivity, repository credentials, proxy settings, and Maven configuration. Based on the error, I identify whether the issue is with Maven, the repository, or the dependency itself and fix it.
**Simple ga gurthupettuko:**
Dependency download avvakapothe → version and repository check cheyyi → network, credentials, proxy verify cheyyi → issue fix cheyyi.
**Eng:** Dependency download fails → check the version and repository → verify network, credentials, and proxy → fix the issue.

---

**3. Maven is showing "Could not transfer artifact". What will you check?**
I first check the repository URL shown in the Maven error and verify whether the Jenkins agent can reach it. Then I check DNS, proxy, firewall, credentials, and repository availability to identify whether the failure is caused by connectivity or authentication.
**Simple ga gurthupettuko:**
Artifact transfer fail ayithe → repository URL chudu → connectivity check cheyyi → credentials and repository availability verify cheyyi.
**Eng:** Artifact transfer fails → check the repository URL → verify connectivity → check credentials and repository availability.

---

**4. Maven dependencies are downloading very slowly. How will you troubleshoot?**
First, I check whether the problem is with the Maven repository or network connectivity. Then I check proxy/DNS issues, repository performance, and whether the Jenkins agent has a local Maven cache. If the agent is newly created, repeated dependency downloads can also increase build time.
**Simple ga gurthupettuko:**
Download slow ayithe → repository and network chudu → proxy/DNS check cheyyi → Maven cache undho verify cheyyi.
**Eng:** Downloads are slow → check the repository and network → verify proxy/DNS → check the Maven cache.

---

**5. Maven works on my local machine but fails in Jenkins. What will you check?**
I compare the local and Jenkins environments, especially Java version, Maven version, JAVA_HOME, settings.xml, environment variables, credentials, network access, and local Maven repository. I also verify that Jenkins is building the expected branch and pom.xml.
**Simple ga gurthupettuko:**
Local work, Jenkins fail → Java/Maven versions compare cheyyi → settings and credentials chudu → branch and environment verify cheyyi.
**Eng:** Local works but Jenkins fails → compare Java and Maven versions → check settings and credentials → verify the branch and environment.

---

**6. Maven works on one Jenkins agent but fails on another. What will you check?**
I compare the working and failing agents for Java/Maven versions, environment variables, permissions, network access, Maven settings, disk space, and local repository. I focus on the agent-specific difference instead of changing the application unnecessarily.
**Simple ga gurthupettuko:**
Oka agent lo work, inkoka agent lo fail → rendu agents compare cheyyi → version, configuration, permissions, network difference find cheyyi.
**Eng:** One agent works and another fails → compare both agents → find the version, configuration, permission, or network difference.

---

**7. Maven says "Dependency could not be resolved". What will you do?**
I check the dependency coordinates and version in pom.xml and verify whether that version exists in the configured repositories. I also check parent POMs, dependency management, mirrors, repository configuration, and any exclusions affecting dependency resolution.
**Simple ga gurthupettuko:**
Dependency resolve avvakapothe → version correct aa chudu → repository lo undho verify cheyyi → POM and dependency management check cheyyi.
**Eng:** Dependency cannot be resolved → check the version → verify it exists in the repository → check the POM and dependency management.

---

**8. Maven is using an old dependency version. What will you check?**
I run mvn dependency:tree to identify which dependency is bringing the old version. Then I check direct dependencies, transitive dependencies, dependency management, and version overrides in the POM.
**Simple ga gurthupettuko:**
Old version vasthe → dependency tree chudu → old version ekkada nunchi vastundo find cheyyi → required version manage cheyyi.
**Eng:** Old version is used → check the dependency tree → find where it comes from → manage the required version.

**9. How do you troubleshoot a Maven dependency conflict?**
I use the dependency tree to identify the conflicting versions and determine which libraries are introducing them. Then I use dependency management or exclusions to control the required version and test the application.
**Simple ga gurthupettuko:**
Dependency conflict ayithe → dependency tree chudu → conflict versions identify cheyyi → correct version ni manage cheyyi.
**Eng:** Dependency conflict occurs → check the dependency tree → identify the conflicting versions → manage the correct version.

---

**10. Maven build is failing because of a Java version mismatch. What will you check?**
I check java -version and mvn -version on the Jenkins agent to see which Java Maven is actually using. Then I verify JAVA_HOME, Jenkins tool configuration, Maven compiler settings, and the Java version required by the application.
**Simple ga gurthupettuko:**
Java mismatch ayithe → java -version and mvn -version chudu → JAVA_HOME and Jenkins Java configuration correct cheyyi.
**Eng:** Java mismatch occurs → check java -version and mvn -version → correct JAVA_HOME and Jenkins Java configuration.

---

**11. Maven build is using the wrong Java version. How will you fix it?**
I first confirm the active Java version using java -version and mvn -version. Then I correct JAVA_HOME or Jenkins tool configuration, ensure Maven uses the required JDK, and rerun the build.
**Simple ga gurthupettuko:**
Wrong Java use avuthe → active Java version check cheyyi → JAVA_HOME correct cheyyi → Maven ki correct JDK set chesi rerun cheyyi.
**Eng:** Wrong Java is used → check the active version → correct JAVA_HOME → configure the required JDK and rerun.

---

**12. Maven tests are failing, but the build itself is successful. What will you check?**
I identify the failed tests from the Maven test reports and console logs. Then I determine whether the issue is related to application code, test data, environment configuration, database connectivity, or an external dependency.
**Simple ga gurthupettuko:**
Build success kani tests fail → failed test chudu → code, data, environment, database issue identify cheyyi → root cause fix cheyyi.
**Eng:** Build succeeds but tests fail → check the failed test → identify the code, data, environment, or database issue → fix the root cause.

---

**13. Maven tests pass locally but fail in Jenkins. What will you check?**
I compare the local and Jenkins environments for Java version, environment variables, test configuration, database connectivity, external services, filesystem paths, and test data. I also check the exact test failure in Jenkins logs.
**Simple ga gurthupettuko:**
Local tests pass, Jenkins fail → environments compare cheyyi → Java, config, database, data check cheyyi → Jenkins logs chudu.
**Eng:** Local tests pass but Jenkins fails → compare the environments → check Java, configuration, database, and data → check Jenkins logs.

---

**14. Maven build is taking much longer than usual. What will you check?**
First, I identify which Maven phase is consuming most of the time. Then I check dependency downloads, test execution, plugins, repository performance, network, agent CPU/memory, and unnecessary build operations.
**Simple ga gurthupettuko:**
Maven slow ayithe → ye phase slow undo chudu → dependencies, tests, plugins check cheyyi → agent resources verify cheyyi.
**Eng:** Maven is slow → find the slow phase → check dependencies, tests, and plugins → verify agent resources.

---

**15. Maven downloads dependencies on every Jenkins build. What will you check?**
I check whether the Jenkins agent has a persistent Maven local repository. If agents are temporary, I check whether a suitable dependency-cache strategy can be used so every build does not download everything again.
**gurthupettuko:**Prathi build dependencies download chesthe → Maven cache undho chudu → reusable cache configure cheyyi.
**Eng:** Dependencies download every build → check the Maven cache → configure a reusable cache.

---

**16. What is causing a Maven local repository issue in Jenkins?**
The local repository can have corrupted files, missing dependencies, permission issues, or insufficient disk space. I check the affected artifact, .m2 repository, disk space, permissions, and agent configuration before cleaning only the required cache.
**Simple ga gurthupettuko:**
.m2 issue ayithe → affected dependency chudu → cache, disk, permissions check cheyyi → required cache matrame clean cheyyi.
**Eng:** .m2 issue occurs → check the affected dependency → verify cache, disk, and permissions → clean only the required cache.

---

**17. Maven authentication is failing with Nexus. What will you check?**
I verify the Nexus repository URL and the credentials configured for Maven. Then I check whether the credentials have the required read or deploy permissions and whether Jenkins is using the correct settings.xml.
**Simple ga gurthupettuko:**
Nexus authentication fail → repository URL chudu → credentials verify cheyyi → settings.xml and permissions check cheyyi.
**Eng:** Nexus authentication fails → check the repository URL → verify credentials → check settings.xml and permissions.

---

**18. Maven cannot upload an artifact to Nexus. What will you check?**
I verify the target repository, repository availability, artifact version, and Maven deployment configuration. Then I check whether the credentials have permission to upload artifacts.
**Simple ga gurthupettuko:**
Artifact upload fail → target repository chudu → deployment config and version check cheyyi → upload permission verify cheyyi.
**Eng:** Artifact upload fails → check the target repository → verify deployment configuration and version → check upload permission.

**19. Maven is uploading the wrong artifact version. What will you check?**
I check the version in pom.xml, parent POM, Maven profiles, and CI/CD parameters. I also verify the Git branch and commit to ensure Jenkins is building the expected source code.
**Simple ga gurthupettuko:**
Wrong version upload → POM version chudu → parent/profile check cheyyi → Git branch and commit verify cheyyi.
**Eng:** Wrong version is uploaded → check the POM version → check parent/profile settings → verify the Git branch and commit.

---

**20. What is the difference between** mvn package **and** mvn install**?**
mvn package compiles, tests, and packages the application into an artifact such as a JAR or WAR. mvn install performs the package lifecycle and also installs the generated artifact into the local Maven repository.
**Simple ga gurthupettuko:**
Package → JAR/WAR create chestundi. Install → JAR/WAR create chesi local Maven repository lo store chestundi.
**Eng:** Package → creates the JAR/WAR. Install → creates it and stores it in the local Maven repository.

---

**21. What is the difference between** mvn package **and** mvn deploy**?**
mvn package creates the application artifact in the local build workspace. mvn deploy performs the build lifecycle and publishes the artifact to the configured remote repository.
**Simple ga gurthupettuko:**
Package → artifact local ga create chestundi. Deploy → artifact ni remote repository ki publish chestundi.
**Eng:** Package → creates the artifact locally. Deploy → publishes the artifact to the remote repository.

---

**22. Maven profile works in DEV but fails in PROD. What will you check?**
I compare the active Maven profiles and environment-specific properties between DEV and PROD. I check endpoints, credentials, configuration values, profile activation, and external dependencies.
**Simple ga gurthupettuko:**
DEV work, PROD fail → active profiles compare cheyyi → properties, credentials, endpoints, dependencies check cheyyi.
**Eng:** DEV works but PROD fails → compare the active profiles → check properties, credentials, endpoints, and dependencies.

---

**23. How do you manage environment-specific configuration in Maven?**
I use Maven profiles where build-time environment differences are required, while keeping sensitive values outside the POM. Secrets should come from secure credential or configuration management rather than being hardcoded.
**Simple ga gurthupettuko:**
DEV/QA/PROD differences → profiles use cheyyi → secrets POM lo hardcode cheyyakunda secure ga manage cheyyi.
**Eng:** DEV/QA/PROD differences → use profiles → keep secrets out of the POM and manage them securely.

---

**24. Maven plugin suddenly stopped working after an update. What will you do?**
I identify the plugin version that changed and compare it with the last successful build. Then I check Java/Maven compatibility and the plugin error, and if necessary I restore a known-good version or apply a compatible fix.
**Simple ga gurthupettuko:**
Plugin update tarvatha fail → changed version identify cheyyi → compatibility check cheyyi → working version ki rollback or fix cheyyi.
**Eng:** Plugin update causes failure → identify the changed version → check compatibility → roll back or apply a compatible fix.

---

**25. Maven build fails because of a plugin execution error. How will you troubleshoot?**
I identify the exact plugin and goal that failed from the Maven logs. Then I check plugin configuration, version, Java/Maven compatibility, required files, and whether the issue happens only on a specific Jenkins agent.
**Simple ga gurthupettuko:**
Plugin execution fail → plugin and goal identify cheyyi → configuration/version check cheyyi → Java/Maven compatibility verify cheyyi.
**Eng:** Plugin execution fails → identify the plugin and goal → check configuration and version → verify Java/Maven compatibility.

---

**26. Maven build succeeds but the generated JAR is missing. What will you check?**
I verify whether the correct lifecycle phase was executed and check the target directory for the generated artifact. I also verify the packaging type and whether another Jenkins step moved or deleted the artifact.
**Simple ga gurthupettuko:**
Build success kani JAR ledu → package phase check cheyyi → target lo search cheyyi → packaging and Jenkins steps verify cheyyi.
**Eng:** Build succeeds but the JAR is missing → check the package phase → search the target directory → verify packaging and Jenkins steps.

---

**27. Maven build fails because the Jenkins agent disk is full. What will you do?**
I check disk usage and identify whether Maven cache, workspace, artifacts, logs, or temporary files are consuming the space. I safely clean unnecessary data and configure proper cleanup or retention to avoid repeated failures.
**Simple ga gurthupettuko:**
Disk full → space ekkuva ekkada use avutundo chudu → cache/workspace/logs clean cheyyi → cleanup policy set cheyyi.
**Eng:** Disk is full → find what is using the space → clean unnecessary cache, workspaces, and logs → configure cleanup.

---

**28. Maven build is successful, but Jenkins cannot find the artifact. What will you check?**
I verify the artifact name, version, packaging type, and output path generated by Maven. Then I check whether the Jenkins archive or publishing step is using the correct artifact path and filename.
**Simple ga gurthupettuko:**
Maven success kani Jenkins artifact dorakakapothe → artifact name/path chudu → version and packaging check cheyyi → Jenkins publish path verify cheyyi.

**Eng:** Maven succeeds but Jenkins cannot find the artifact → check the artifact name and path → verify version and packaging → check the Jenkins publish path.

---

**29. How do you make Maven builds reproducible?**
I use fixed dependency and plugin versions and maintain consistent Java and Maven versions across build agents. I also keep build configuration controlled so that the same source code produces a predictable result.
**Simple ga gurthupettuko:**
Same code ki same build kavali → dependency/plugin versions fix cheyyi → Java/Maven versions control cheyyi → same configuration maintain cheyyi.
**Eng:** Same code should produce the same build → fix dependency/plugin versions → control Java/Maven versions → maintain consistent configuration.

---

**30. Maven build fails because of a SNAPSHOT dependency. What will you check?**
I verify whether the required SNAPSHOT version exists in the configured snapshot repository. Then I check repository configuration, credentials, network connectivity, and whether Maven is resolving the expected snapshot version.
**Simple ga gurthupettuko:**
SNAPSHOT fail → snapshot repository lo version undho chudu → repository/configuration check cheyyi → credentials and resolution verify cheyyi.
**Eng:** SNAPSHOT fails → check whether the version exists in the snapshot repository → verify configuration → check credentials and resolution.

---

**31. Maven dependency works in one project but not another. What will you compare?**
I compare both projects' POM files, dependency versions, parent POMs, repositories, profiles, dependency management, and exclusions. This helps identify why one project resolves the dependency differently.
**Simple ga gurthupettuko:**
Oka project lo work, inkoka project lo fail → rendu POMs compare cheyyi → versions, repositories, profiles, exclusions check cheyyi.
**Eng:** One project works and another fails → compare both POMs → check versions, repositories, profiles, and exclusions.

---

**32. Maven build is passing, but a vulnerable dependency is reported. What will you do?**
I identify the vulnerable dependency and check whether it is direct or transitive using the dependency tree. Then I upgrade to a safe compatible version or replace/exclude the vulnerable dependency after testing the application.
**Simple ga gurthupettuko:**
Build pass kani vulnerability vasthe → vulnerable dependency identify cheyyi → direct/transitive check cheyyi → safe version ki upgrade cheyyi.
**Eng:** Build passes but a vulnerability appears → identify the dependency → check direct or transitive → upgrade to a safe version.

---

**33. How do you prevent vulnerable dependencies from reaching Production?**
I integrate dependency and security scanning into the CI/CD pipeline and define policies for vulnerable dependencies. Critical issues can be used to block the build or deployment until they are resolved or formally approved.
**Simple ga gurthupettuko:**
Vulnerable dependency Production ki vellakudadhu → pipeline lo scan run cheyyi → policy enforce cheyyi → critical issue unte deployment block cheyyi.
**Eng:** Vulnerable dependencies should not reach Production → run scanning in the pipeline → enforce policies → block deployment for critical issues.

---

**34. Maven suddenly starts downloading a large number of dependencies. What will you check?**
I check whether the local Maven repository was deleted or the Jenkins agent was replaced. I also verify whether dependency versions, settings.xml, repositories, or the POM changed recently.
**Simple ga gurthupettuko:**
Sudden ga dependencies ekkuva download ayithe → local cache check cheyyi → agent change chudu → POM and settings changes verify cheyyi.
**Eng:** Many dependencies suddenly download → check the local cache → check for agent changes → verify POM and settings changes.

---

**35. Maven build works with one** settings.xml **but fails with another. What will you check?**
I compare the repository URLs, mirrors, credentials, proxy configuration, profiles, and active settings between the two files. I identify which setting is changing dependency resolution or authentication.
**Simple ga gurthupettuko:**
One settings work, another fail → rendu settings compare cheyyi → repository, mirror, credentials, proxy check cheyyi.
**Eng:** One settings file works and another fails → compare both files → check repository, mirror, credentials, and proxy settings.

---

**36. Maven build is failing only when connected to the corporate network. What will you check?**
I check proxy configuration, DNS, firewall rules, repository access, and whether Maven is required to use a corporate proxy. I compare network behavior with a working environment to identify what is blocking repository access.
**Simple ga gurthupettuko:**
Corporate network lo matrame fail → proxy and DNS check cheyyi → firewall/repository access verify cheyyi → Maven proxy configuration chudu.
**Eng:** Failure occurs only on the corporate network → check proxy and DNS → verify firewall and repository access → check Maven proxy configuration.

**37. Maven build succeeds, but the artifact is not available in Nexus. What will you check?**
I verify whether the deployment step actually executed after the package phase. Then I check the target Nexus repository, artifact coordinates, version, deployment logs, and upload permissions.
**Simple ga gurthupettuko:**
Build success kani Nexus lo artifact ledu → deploy step run ayinda chudu → repository, version, logs, permissions check cheyyi.
**Eng:** Build succeeds but the artifact is missing in Nexus → check whether deploy ran → verify repository, version, logs, and permissions.

---

**38. Maven build is failing after a** pom.xml **change. What will you check?**
I compare the current pom.xml with the last working version and identify what changed. Then I check dependency versions, plugin configuration, repositories, profiles, exclusions, and XML syntax before rerunning the build.
**Simple ga gurthupettuko:**
POM change tarvatha fail → old POM tho compare cheyyi → changed dependency/plugin/config identify chesi fix cheyyi.
**Eng:** Build fails after a POM change → compare it with the old POM → find the changed dependency, plugin, or configuration and fix it.

---

**39. How do you troubleshoot Maven build failure in a Production CI/CD pipeline?**
First, I identify the failed Maven phase and exact error from the Jenkins logs. Then I verify dependencies, Java/Maven versions, repository access, credentials, POM changes, and agent configuration before rerunning the pipeline.
**Simple ga gurthupettuko:**
Production build fail → failed Maven phase chudu → logs check cheyyi → dependency/repository/version/config verify cheyyi → fix chesi rerun cheyyi.
**Eng:** Production build fails → check the failed Maven phase → check the logs → verify dependencies, repository, versions, and configuration → fix and rerun.

---

**40. How would you optimize Maven build time in Jenkins?**
I first identify the slowest Maven phase and determine whether the delay comes from dependencies, tests, plugins, repository access, or the build agent. Then I optimize caching, remove unnecessary work, and use parallel execution where it is safe.
**Simple ga gurthupettuko:**
Maven slow → slow phase identify cheyyi → dependencies/tests/plugins reason find cheyyi → cache and safe parallel execution use cheyyi.
**Eng:** Maven is slow → identify the slow phase → find the dependency, test, or plugin issue → improve caching and use safe parallel execution.